

EduGenie: Google Gemini Powered Learning Assistant

Project Description:

EduGenie is a lightweight AI-powered educational assistant that harnesses Generative AI to simplify and enhance the learning experience for students, self-learners, and educators. The platform helps users understand concepts, answer questions, generate quizzes, summarize learning materials, and receive personalized learning-path recommendations, making learning more interactive, accessible, and efficient across academic levels and subject domains.

Built with FastAPI for the backend and a responsive HTML + CSS frontend, EduGenie combines lightweight local logic with the Google Gemini API to deliver fast, accurate educational assistance. Its optimized, modular architecture allows it to run efficiently even on resource-constrained devices such as the Mac M1, with secure API key management via .env, making it well-suited for both personal learning and academic/internship projects.

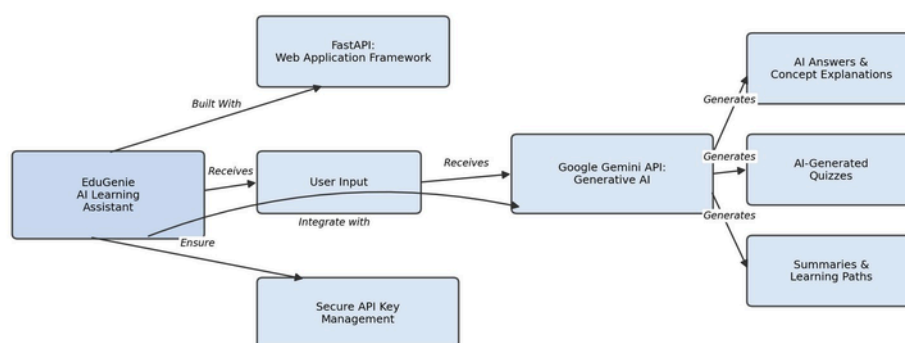
Scenarios

Scenario 1: A student wants to learn about oceans and rivers. They ask, “Which is the largest ocean?” EduGenie instantly provides an accurate answer along with additional educational context.

Scenario 2: A student studying the Pythagoras Theorem wants to assess their understanding. They select “Generate Quiz,” and EduGenie creates topic-specific questions for self-evaluation.

Scenario 3: A learner interested in SQL requests a structured learning roadmap. EduGenie generates a personalized learning path covering beginner, intermediate, and advanced topics with study recommendations and progression guidance.

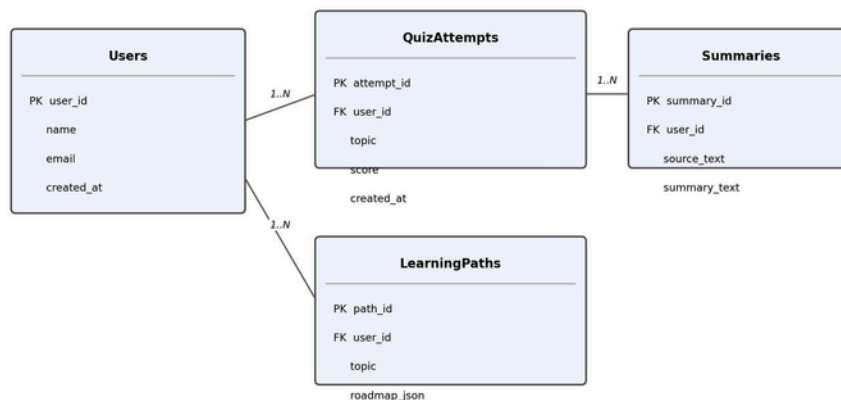
Technical Architecture:



Description: EduGenie uses a modular three-tier architecture to deliver fast, AI-driven educational assistance. The frontend provides a responsive user interface built with HTML and CSS, while the FastAPI-based backend handles routing, input validation, and prompt construction. It interfaces with the Google Gemini API to generate answers, explanations, quizzes, summaries, and learning paths, with secure API key management and local/cloud deployment options.

Database & ER Diagram:

EduGenie uses a relational database (SQL / PostgreSQL / SQLite) to persist quiz attempts, learning-path history, and generated summaries per user. The core entities and their relationships are shown below.



Pre-requisites:

- **FastAPI Framework Knowledge:** FastAPI Documentation (<https://fastapi.tiangolo.com>)
- **Google Gemini API Familiarity:** Google AI for Developers Documentation (<https://ai.google.dev/docs>)
- **HTML, CSS, and JavaScript Skills:** W3Schools HTML/CSS/JavaScript Tutorials
- **Python Programming Proficiency:** Python Documentation (<https://docs.python.org>)
- **Version Control with Git:** Git Documentation (<https://git-scm.com/doc>)
- **Development Environment Setup:** Uvicorn / FastAPI Installation Guide
- **Database Basics:** SQLAlchemy / SQLite Documentation

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate a Google Gemini API key and configure it securely in the project environment.
- **Activity 1.2:** Research and select the appropriate Gemini model for educational content generation.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the frontend, backend, and AI integration.
- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for FastAPI and the Gemini API.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core functionalities: Question Answering, Concept Explanation, Quiz Generation, Summarization, and Learning Path Recommendation.
- **Activity 2.2:** Implement the FastAPI backend to manage routing and user input processing, ensuring smooth API interactions.

Milestone 3: main.py Development

- **Activity 3.1:** Write the main application logic in main.py, establishing routes for each feature and integrating AI responses.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using HTML, CSS, and JavaScript, ensuring a responsive and intuitive layout.

- **Activity 4.2:** Create dynamic templates with Jinja2 to display results based on user interactions.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring the server environment and ensuring all dependencies are correctly installed.
- **Activity 5.2:** (Optional) Deploy the application publicly using a hosting provider (e.g., Render, Railway) to expose it over a secure public URL.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate Generative AI model from Google Gemini for EduGenie's learning-assistance needs. This involves researching the capabilities of available Gemini models, ensuring the chosen model aligns with the application's objectives of answering questions, explaining concepts, generating quizzes, summarizing material, and recommending learning paths.

Activity 1.1: Generate a Google Gemini API Key

Before the application can communicate with the Gemini LLM, you need a valid API key. Follow the steps

below to create one and connect it securely to the project.

- **Step 1 – Create a Google AI Studio Account:** Visit <https://aistudio.google.com> and sign in with your Google account.
- **Step 2 – Navigate to API Keys:** Click “Get API Key” from the left navigation menu, or go directly to the API keys page.
- **Step 3 – Create a New API Key:** Click “Create API Key” and select or create a Google Cloud project to associate it with.
- **Step 4 – Copy the API Key:** Copy the generated key immediately and store it in a safe place.
- **Step 5 – Add the Key to Your Project:** Create a .env file in the root of your project directory and paste the key as shown below:

```
GOOGLE_GEMINI_API_KEY=your_copied_api_key_here
```

- **Step 6 – Verify the Key is Loaded:** The main.py file loads this key automatically at startup using python-dotenv:

```
from dotenv import load_dotenv
import os

load_dotenv()
GEMINI_API_KEY = os.environ.get("GOOGLE_GEMINI_API_KEY")
```

- **Important – Never Commit Your API Key:** Add .env to your .gitignore file to prevent accidentally pushing the key to a public repository:

```
.env
```

Activity 1.2: Research and Select the Appropriate Generative AI Model

Here are the things that needed to be researched to select the best model for the project:

- **Understand the Project Requirements:** Review EduGenie's specific needs — the types of content it will generate (answers, explanations, quizzes, summaries, learning paths).
- **Explore Gemini's Model Documentation:** Examine the available Gemini models, including their functionalities, context windows, speed, and limitations.
- **Evaluate Model Performance:** Compare models based on response quality, generation speed, and relevance to educational content tasks.

- **Select the Optimal Model:** Choose gemini-1.5-flash for its balance of response quality and low latency, suitable for near-real-time educational Q&A and content generation.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including the frontend (HTML, CSS, JavaScript), FastAPI backend, and Gemini API integration (see Technical Architecture section above).
- **Detail Frontend Functionality:** Outline how users interact with the application — asking a question, requesting a concept explanation, generating a quiz, summarizing text, or requesting a learning path.
- **Outline Backend Responsibilities:** Specify how the FastAPI backend handles incoming POST requests to each feature route, validates and sanitizes user input, builds a structured prompt, and communicates with the Gemini API.
- **Describe AI Integration Points:** Define how the backend constructs prompts using feature-specific templates, calls the Gemini API, parses the response, and returns structured results to the frontend.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.10+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
D:\projects>python -m venv venv
D:\projects>"d:\projects\venv\Scripts\activate"
(venv)D:\projects> pip install -rrequirements.txt
```

- **Install FastAPI & Uvicorn:** Use pip to install the web framework and ASGI server:

```
(venv)D:\projects> pip install fastapi==0.111.0 uvicorn==0.30.0
```

- **Install Google Generative AI SDK:** Install the Python library to interact with the Gemini API:

```
(venv)D:\projects> pip install google-generativeai==0.7.0
```

- **Install python-dotenv:** Install dotenv support for secure API key management:

```
(venv)D:\projects> pip install python-dotenv==1.0.1
```

- **Set Up Application Structure:** Create the project directory structure including folders for routes, services, models, db, templates, and static files:

```
/EduGenie-root
/app
main.py
/routes          (qa.py, concept.py, quiz.py, summarize.py, learning_path.py)
/services        (gemini_client.py, prompt_templates.py)
/models          (schemas.py)
/db              (database.py, models.py)
/templates
/static          (css, js)
.env
.gitignore
requirements.txt
```

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

Implement each core feature as an independent FastAPI route, driven by a feature-specific structured prompt sent to the Gemini API.

- **Question Answering:** Accepts a user question and returns an accurate answer with additional educational context.
- **Concept Explanation:** Breaks down a requested concept into a simplified, easy-to-understand explanation.
- **Quiz Generator:** Returns a set of topic-specific quiz questions (with answers) for self-evaluation.
- **Text Summarizer:** Condenses submitted learning material into a concise summary.
- **Learning Path Recommender:** Returns a structured beginner → intermediate → advanced roadmap for a requested topic.

Activity 2.2: Implement the FastAPI Backend

Define Routes in FastAPI:

- Set up routers in `main.py` for the home page and each content-generation endpoint (`/ask`, `/explain`, `/quiz`, `/summarize`, `/learning-path`).

Process User Input:

- Create a form in `index.html` for users to submit their question, topic, or text, along with the selected feature.
- Use FastAPI's Pydantic request models to parse and validate JSON data submitted from the frontend via Fetch/AJAX.
- Validate that the input text is non-empty and within a reasonable character limit before processing.

Integrate API Calls:

- Within each route handler, call the relevant `build_prompt()` function to construct a structured prompt using the feature-specific template.
- Submit the prompt to the Gemini API using the `google-generativeai` SDK with the `gemini-1.5-flash` model.
- Parse and validate the response, handling markdown/JSON formatting variations returned by the model.

Milestone 3: main.py Development

Milestone 3 focuses on creating and refining the core logic of the `main.py` file, which serves as the backbone of the EduGenie application. In this milestone, you'll set up each feature's functionality through FastAPI routes, establish reliable prompt construction and API interaction, and conduct testing to ensure each feature performs as expected.

Activity 3.1: Writing the Main Application Logic in `main.py`

Define the Core Routes in `main.py`:

- `/` — Home page route that renders the main `index.html` interface

```
@app.get("/")
def index(request: Request):
    return templates.TemplateResponse("index.html", {"request": request})
```

- `/ask` — POST endpoint that accepts JSON input, invokes the AI, and returns a structured answer

```
@app.post("/ask")
async def ask_question(payload: QuestionRequest):
    question = payload.question.strip()

    if not question:
        raise HTTPException(status_code=400, detail="Question is required.")
    if len(question) > 2000:
        raise HTTPException(status_code=400, detail="Input too long. Keep it under 2000 characters.")
```

```

try:
    prompt = build_qa_prompt(question)
    response = gemini_model.generate_content(prompt)
    result = extract_json(response.text)

    if "answer" not in result:
        raise ValueError("Invalid response structure from AI")

    return {
        "success": True,
        "answer": result["answer"],
        "context": result.get("context", ""),
    }
except ValueError as e:
    raise HTTPException(status_code=500, detail=f"Failed to parse AI response: {e}")
except Exception as e:
    raise HTTPException(status_code=500, detail=f"Generation failed: {e}")

```

Set Up Route Handlers for Each Feature:

- For each route (/ask, /explain, /quiz, /summarize, /learning-path), implement logic that reads and validates the incoming JSON body using a Pydantic model.
- Apply input validation: return a 400 error if the required field is empty or exceeds the character limit.
- Route Fetch/AJAX requests from the frontend to the appropriate endpoint, which processes the data and returns a JSON response.

Define Prompt Engineering Helpers:

- **FEATURE_INSTRUCTIONSdictionary**: maps each feature (qa, explain, quiz, summarize, learning_path) to detailed instructions embedded in the prompt.

```

FEATURE_INSTRUCTIONS = {
    "qa": "Answer the question accurately and concisely, then add 1-2 sentences of "
        "useful educational context.",
    "explain": "Explain the concept in simple, easy-to-understand language suitable "
        "for a student encountering it for the first time.",
    "quiz": "Generate 5 topic-specific multiple-choice questions with 4 options each "
        "and the correct answer marked.",
    "summarize": "Summarize the provided text into concise, exam-ready study notes.",
    "learning_path": "Create a structured roadmap with beginner, intermediate, and "
        "advanced topics and study recommendations for the given subject.",
}

```

- **build_prompt()**: assembles a structured prompt string from the user's input and the feature instruction, instructing the model to return strictly valid JSON.

```

def build_prompt(feature: str, user_input: str) -> str:
    instruction = FEATURE_INSTRUCTIONS.get(feature, "Respond helpfully.")
    return f"""You are an expert educational assistant.

{instruction}

User input: {user_input}

Respond ONLY with valid JSON, no markdown formatting."""

```

- **extract_json()**: a robust parser that first attempts `json.loads()`, then strips markdown code fences, and finally uses a regex to locate a raw JSON object, ensuring reliable parsing across model response variations.

```

def extract_json(text: str) -> dict:
    """Extract JSON from LLM response, handling markdown code blocks."""
    try:

```

```

        return json.loads(text)
    except json.JSONDecodeError:
        pass

    match = re.search(r'```(?:json)?\s*([^\s]*)```', text)
    if match:
        try:
            return json.loads(match.group(1).strip())
        except json.JSONDecodeError:
            pass

    match = re.search(r'\{[^\s]*\}', text)
    if match:
        try:
            return json.loads(match.group(0))
        except json.JSONDecodeError:
            pass

    raise ValueError("Could not extract valid JSON from response")

```

Milestone 4: Frontend Development

Milestone 4 focuses on developing a clean, user-friendly interface for EduGenie. This involves designing a responsive HTML and CSS layout to enhance usability, and creating dynamic content rendering with vanilla JavaScript to display AI-generated answers, quizzes, summaries, and learning paths.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop the main HTML file (index.html) as the single-page interface, including a header with the EduGenie brand and tagline, a feature-selector, an input section, and a results section.
- Use semantic HTML elements to keep the structure organized and accessible for all users.

Design a Responsive Layout Using CSS:

- Create a CSS stylesheet (static/css/style.css) implementing a clean, student-friendly aesthetic.
- Use flexbox/grid layouts to organize the results panel (answer/explanation on one side, quiz/learning-path cards on the other, depending on the selected feature).
- Add media queries to ensure the layout adapts across desktop, tablet, and mobile screen sizes.

Create Separate UI Components for Each Output Type:

- **Answer/Explanation Card:** displays the AI's answer or concept explanation with supporting context.
- **Quiz Card:** renders each generated question with selectable options and immediate self-check feedback.
- **Summary Card:** displays the condensed study notes in a clean, readable format.
- **Learning Path Timeline:** displays the beginner → intermediate → advanced roadmap as a step list.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

Handle Form Submission Asynchronously:

- Attach a submit event listener to the generator form that prevents default form submission and sends the data as a JSON POST request to the relevant endpoint via the Fetch API.
- During loading, disable the submit button and show a loading indicator in place of the button text.

Render Results Dynamically:

- Implement a `renderResults(data)` function in `main.js`, which dynamically creates and injects DOM elements for the returned content into the results container.
- After rendering, automatically smooth-scroll the viewport to the results section using `scrollIntoView()`.

Restore UI State After Generation:

- In the final block of the async handler, re-enable the submit button and restore the original button text regardless of success or failure.

Milestone 5: Deployment

In Milestone 5, the focus is on deploying EduGenie first locally to verify correct operation, and then optionally to a hosting provider to make it accessible over a public URL. This involves configuring the server environment, managing dependencies, and launching the app.

Activity 5.1: Preparing the Application for Local Deployment

- **Create and activate a virtual environment, then install dependencies:**

```
D:\projects>python -m venv venv
D:\projects>"d:\projects\venv\Scripts\activate"
(venv)D:\projects>pip install -rrequirements.txt
```

- **Configure Environment Variables:** Create a .env file in the project root and add your Gemini API key. The app loads this automatically using python-dotenv at startup:

```
GOOGLE_GEMINI_API_KEY=your_gemini_api_key_here
```

Activity 5.2: Local Testing and Verification

- **Start the FastAPI Development Server:** Run the application locally using Uvicorn:

```
(venv)D:\projects>uvicorn app.main:app --reload
```

- **Access the App:** Once running, open a browser and navigate to <http://127.0.0.1:8000> to access EduGenie.
- **Verify All Features:** Test each feature — Question Answering, Concept Explanation, Quiz Generation, Summarization, and Learning Path Recommendation — to confirm AI responses are correctly generated, parsed, and rendered.

Activity 5.3: (Optional) Public Deployment

For sharing EduGenie beyond a local machine, the FastAPI app can be deployed to a hosting provider such as Render, Railway, or Azure App Service, or exposed temporarily via a tunneling tool. The exact steps depend on the chosen platform; in general this involves:

- Pushing the project to a Git repository (GitHub).
- Connecting the repository to the hosting provider and configuring the start command (e.g., `uvicorn app.main:app --host 0.0.0.0 --port $PORT`).
- Setting the `GOOGLE_GEMINI_API_KEY` as a secure environment variable on the hosting platform (never committed to the repository).
- Verifying the deployed application at the platform-provided public URL.

Milestone 6: Conclusion

EduGenie is a lightweight, Generative-AI-powered educational assistant built with FastAPI and styled with a clean, responsive HTML/CSS interface. It uses the Google Gemini API to instantly provide accurate answers, simplified concept explanations, self-assessment quizzes, concise summaries, and personalized learning paths across academic levels and subject domains. The project — covering model selection, backend/frontend development, and deployment — demonstrates how a focused AI integration and a modular architecture can meaningfully support students, self-learners, and educators, while remaining efficient enough to run on resource-constrained devices such as the Mac M1.

Looking ahead, EduGenie offers room for expansion, such as adding user authentication and progress tracking, multi-language support, richer analytics on quiz performance, and deeper integration between the summarization and learning-path features for a more connected study experience. By continuing to refine prompt engineering and frontend accessibility, EduGenie is well-positioned to grow from a focused internship project into a more comprehensive personal learning companion.

Exploring the Website's Web Pages:

Home / Generator Page:

Description: The single-page interface of EduGenie serves as both the landing page and the interaction tool. It features a clean, student-friendly design with a header displaying the EduGenie brand and tagline, a feature selector (Ask a Question, Explain a Concept, Generate Quiz, Summarize Text, Learning Path), an input section, and a results panel — all on one seamless page.

Input Section:

Description: A simple, focused form containing the main input area. Users select which feature they want (Q&A, Explanation, Quiz, Summary, or Learning Path), enter their question, topic, or text (up to 2000 characters), and click the Generate button. While the AI processes the request, the button transitions to a loading state.

Results Section:

Description: Revealed dynamically after generation, this section displays the AI output in a layout appropriate to the selected feature — an answer/explanation card, a set of quiz question cards, a summary card, or a learning-path timeline. The viewport automatically smooth-scrolls to this section upon content generation.

